Aim: To develop a blockchain-powered web application using Solidity programming language on Remix IDE and embedding Metamask Wallet

Tasks Performed:

- Sent 0.01 SepoliaETH from Account 1 to Account 2 in MetaMask and checked both balances.
- Verified the transfer on Sepolia Etherscan.
- Compiled the TRUSTEscrow contract in Remix with the environment set to "Sepolia Testnet – MetaMask".
- Deployed it and confirmed the deployment in MetaMask. The Remix console showed the contract as verified.
- Sent 1 ETH into the contract and read the escrow balance with `getEscrowBalance`, which returned 1 ETH.

Theory:

Blockchain Technology – A distributed, immutable ledger maintained across a network of nodes. Each block holds transactions, a timestamp and the hash of the previous block, making the chain tamper-evident. Once confirmed by consensus, data cannot be altered or deleted.

Smart Contracts – Self-executing programs deployed on a blockchain. Each contract has its own address, stores data in state variables and exposes functions that anyone can call. Any function that changes data is a transaction requiring gas and a signature from the caller's wallet.

Solidity – A statically-typed, contract-oriented language for Ethereum-compatible blockchains. It supports structs, mappings, modifiers, events and access control, and is compiled into EVM bytecode before deployment.

Remix IDE – A free, browser-based IDE to write, compile, debug and deploy Solidity contracts. Its "Deploy & Run Transactions" module can target the in-browser Remix VM or a real network through an injected wallet such as MetaMask.

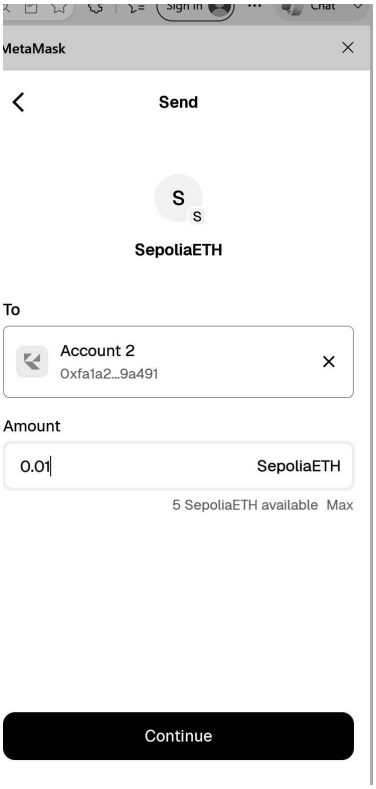
MetaMask Wallet – A browser-extension wallet that stores private keys locally. It acts as an injected provider between a dApp and the blockchain, showing the network, gas fee and destination for every transaction and requiring explicit user confirmation before signing.

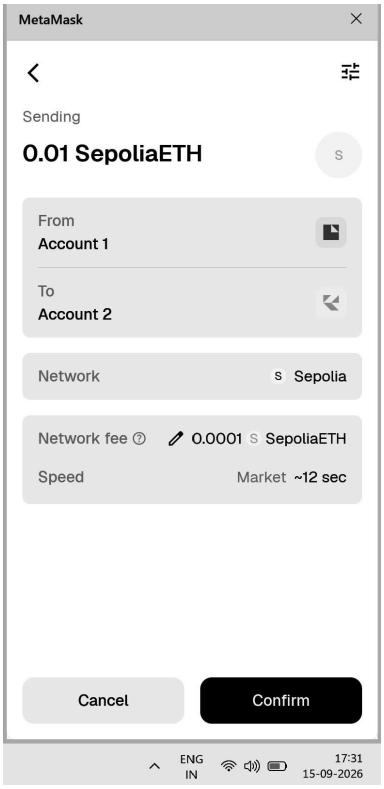
Testnet and Gas – A testnet such as Sepolia mirrors Ethereum mainnet but uses worthless test ETH obtained from faucets. Gas is the fee paid to execute transactions, allowing contracts to be tested without financial risk.

Code & Output:

Step 1 — Fund the second account using MetaMask (Sepolia network)

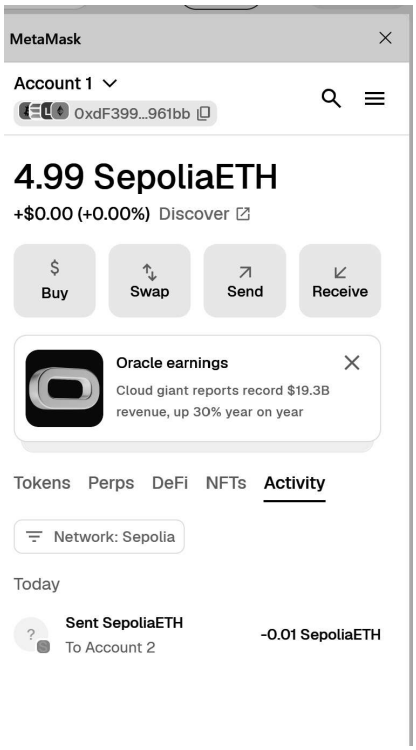
0.01 SepoliaETH is sent from Account 1 to Account 2 and the network fee is reviewed before confirming.

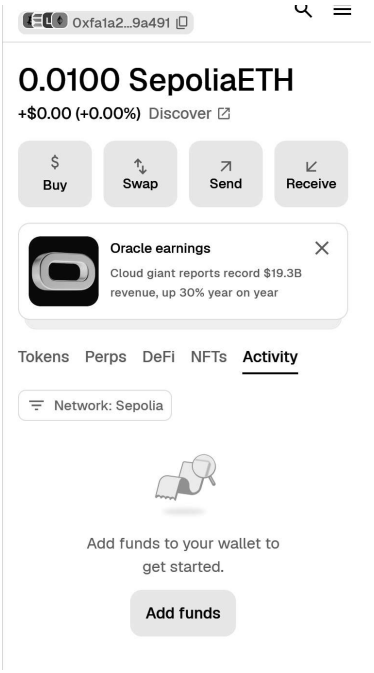




Step 2 — Check the balances after the transfer

Account 1 shows 4.99 SepoliaETH and the receiving account (Ved Joshi) shows 0.0100 SepoliaETH.

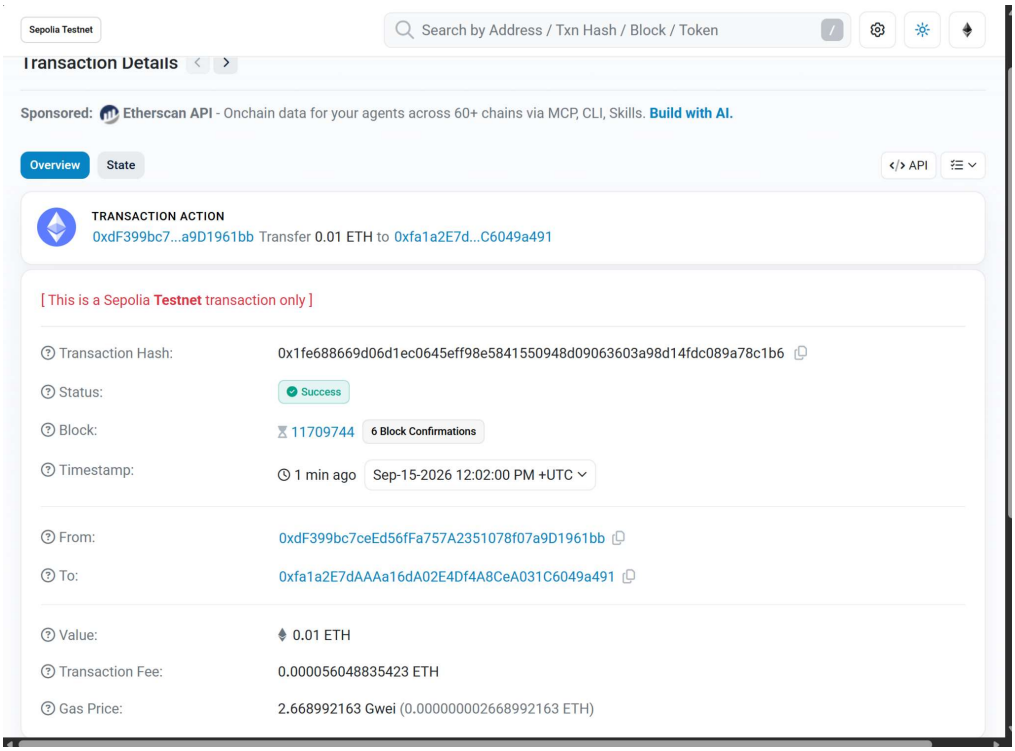




Step 3 — Verify the transaction on Etherscan

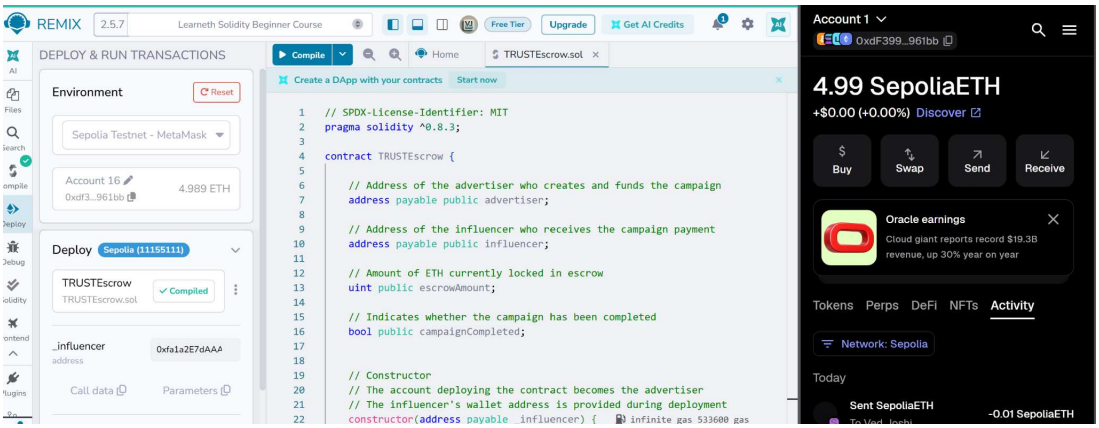
https://sepolia.etherscan.io

The transfer appears with status Success, its transaction hash, block number and gas fee.



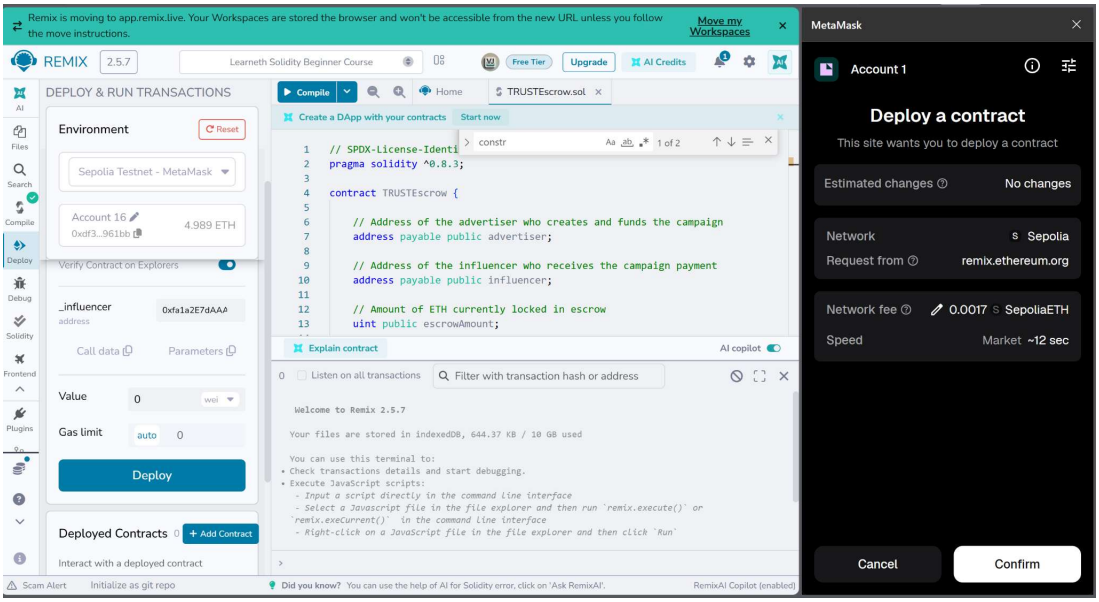
Step 4 — Compile the contract and connect Remix to MetaMask

The TRUSTEscrow contract (Solidity ^0.8.3) is compiled in Remix with the environment set to "Sepolia Testnet – MetaMask". The deploying account becomes the advertiser and the influencer's wallet address is passed to the constructor.



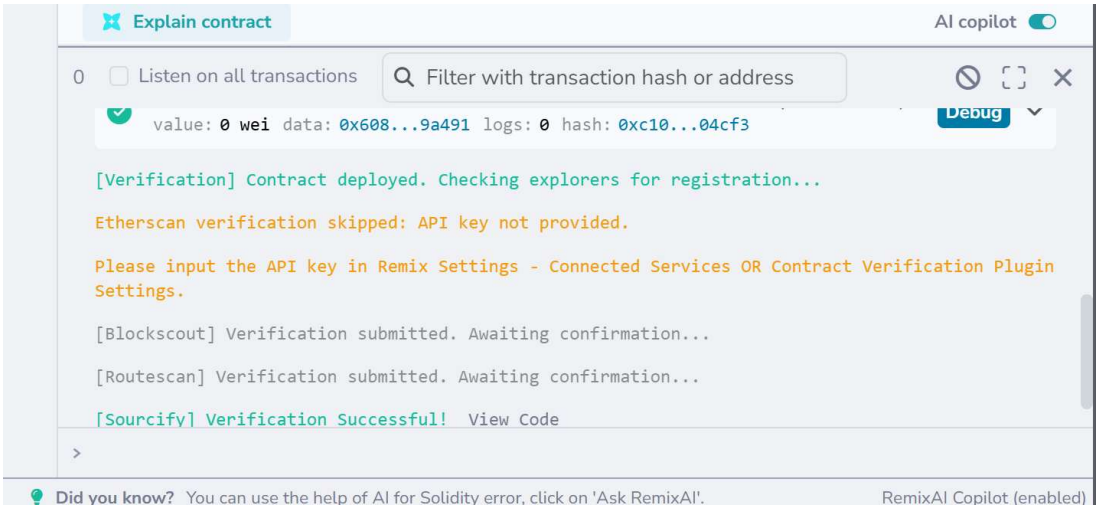
Step 5 — Deploy the contract and confirm in MetaMask

Clicking Deploy raises a “Deploy a contract” request in MetaMask showing the Sepolia network, the request origin (remix.ethereum.org) and the estimated network fee; it is signed with Confirm.



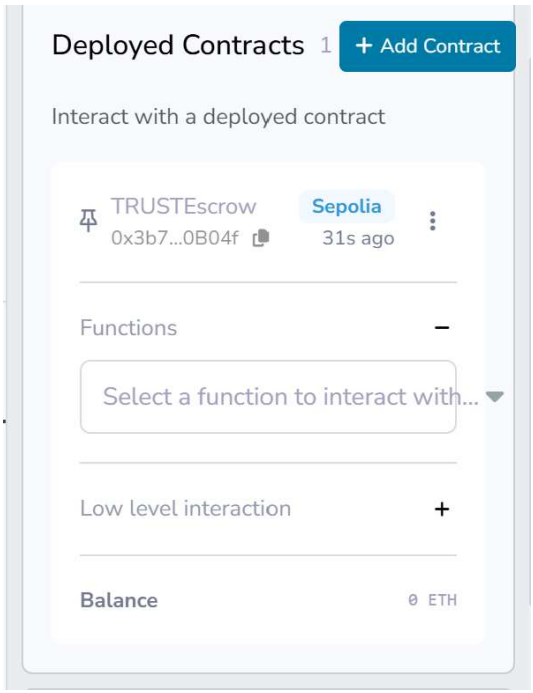
Step 6 — Verify the deployment in the Remix console

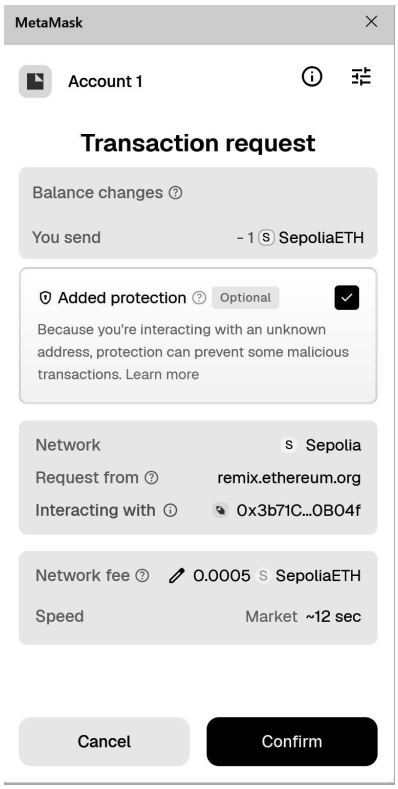
The console shows the successful deployment transaction and the contract verification status on the explorers (Sourcify verification successful).



Step 7 — Interact with the deployed contract and lock funds

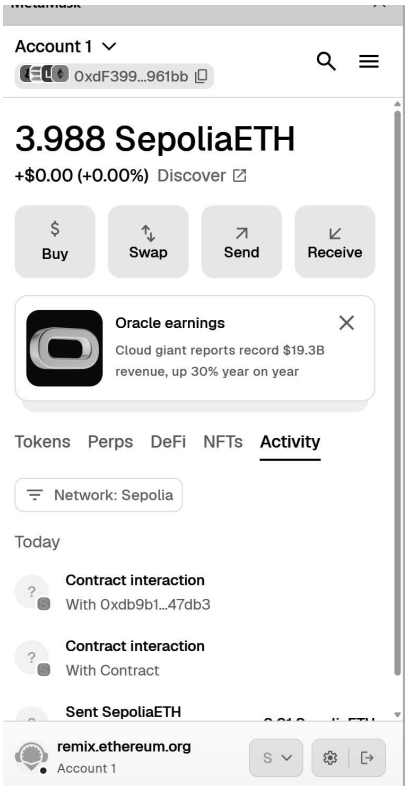
TRUSTEscrow appears under Deployed Contracts on Sepolia. A transaction sending 1 SepoliaETH to the contract is raised and confirmed through MetaMask.

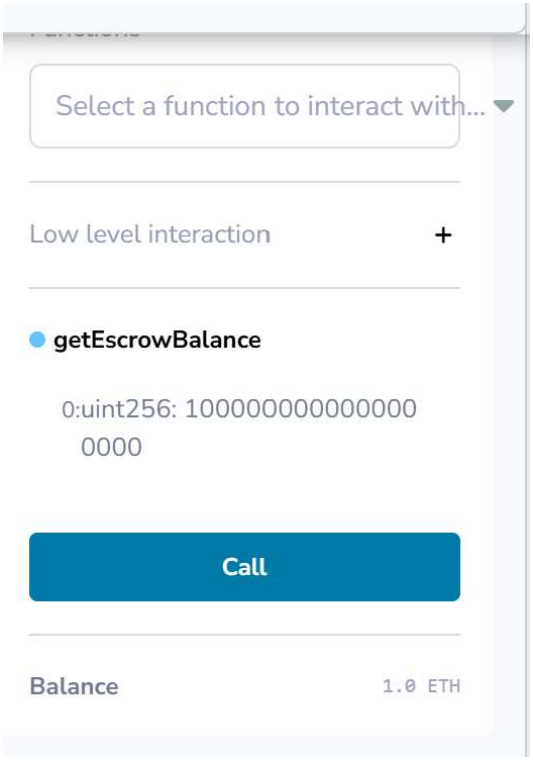




Step 8 — Read the escrow balance

After confirmation, the Account 1 balance drops to 3.988 SepoliaETH and the contract interactions are listed in MetaMask. Calling **getEscrowBalance** returns 1000000000000000000 wei (1 ETH) and the contract balance shows 1.0 ETH.





Conclusion:

The **TRUSTEscrow** smart contract was successfully compiled, deployed and verified on the **Sepolia Testnet** using Remix IDE integrated with MetaMask. Funds were locked in the contract through a wallet-signed transaction and the escrow balance was read back correctly, with the transactions visible on Etherscan. The experiment demonstrated the complete workflow of **wallet-based transaction signing, contract deployment, and on-chain data storage and retrieval**.